Audio AI Systems in Dragon's Hollow

Applied AI – Assignment Writeup

Project Overview

Dragon's Hollow is a mobile web app companion for a live D&D campaign. Players use it to chat with AI-powered NPCs in real time. This document covers the three audio AI subsystems: **speech-to-text input**, **text-to-speech NPC voice output**, and the **prompt engineering layer** that controls what NPCs say and how they sound.

---

1. Pretrained Foundation Models

1a. Text-to-Speech — Google Gemini 2.5 Flash TTS

Model: `gemini-2.5-flash-preview-tts`
Provider: Google DeepMind
Type: Large-scale neural TTS model with 30 prebuilt voices

Gemini TTS is a foundation model trained on large corpora of human speech. It accepts natural language text and delivery cue annotations (described below) and returns raw 24kHz mono 16-bit PCM audio. The app wraps this in a WAV container and serves it to the client.

Each NPC has a dedicated voice selected from the model's 30 prebuilt voices (e.g., Enceladus, Fenrir, Aoede, Charon), chosen to match the character's personality and demographics. Voice assignments are stored in per-character config files alongside the accent prompt strings.

Why not Whisper/SpeechT5? Gemini TTS produces significantly more natural, expressive speech than SpeechT5 for long-form character dialogue, and avoids the self-hosting complexity of running a Whisper-sized model for inference. The professor confirmed the choice of model is at the student's discretion.

1b. Speech-to-Text — Web Speech API (Google Speech Recognition)

Model: Google Cloud Speech-to-Text (accessed via the browser's `SpeechRecognition` / `webkitSpeechRecognition` API)
Type: Deep neural network ASR model, architecturally similar to Whisper

The app uses the browser-native Web Speech API for the "Speech to Text" button in the message keyboard. When activated, it streams audio from the device microphone and returns interim and final transcript results in real time. The transcript is injected directly into the chat input field.

Configuration:

- `continuous: false` — captures a single utterance
- `interimResults: true` — shows real-time partial transcripts while the user speaks
- `lang: 'en-US'` — English language model

The browser delegates this to Google's production ASR infrastructure, which uses the same class of models as Whisper (encoder-decoder transformer trained on large multilingual speech datasets).

1c. NPC Dialogue — OpenAI GPT-4o-mini

Model: `gpt-4o-mini`
Role: Generates in-character NPC responses that the TTS model then speaks aloud

While not strictly an audio model, GPT-4o-mini is the upstream component whose output quality directly determines what the TTS model receives. The NPC dialogue system conditions each response on character personality documents, campaign memory, and location context (detailed in Section 2).

---

2. Prompt and Input Engineering

The system has three distinct layers of prompt engineering that work together.

2a. NPC Character System Prompts

Each NPC is defined by a set of plain-text documents loaded at inference time:

File	Purpose
`SOUL.md`	Core personality, backstory, speech patterns, values, fears
`CONTEXT.md`	Campaign-specific lore and NPC's role in the world
`MEMORY.md`	Long-term facts the NPC has accumulated over sessions
`journal.md`	Recent events, updated automatically after every N messages

These are concatenated into a structured system prompt:

```
You ARE {characterName}. You speak in first person. You NEVER refer to yourself in
third person...

## YOUR CHARACTER
{soul}

## CAMPAIGN CONTEXT
{context}

## YOUR MEMORIES
{memory}

## RECENT EVENTS (from your journal)
{journal}

## CURRENT LOCATION
You are currently at: {locationName} — {locationDescription}
```

```
## INSTRUCTIONS
- Keep it SHORT — 1-2 sentences most of the time. Think casual chat, not
paragraphs.
- Respond as your character would, with their voice and mannerisms
...
```

This approach encodes character identity as structured prompt context rather than fine-tuning, which means new NPCs can be added without retraining.

2b. Voice Mode Instructions (TTS-Aware Prompting)

When a message is prefixed with `!voice`, the server appends a special **Voice Mode** block to the NPC's system prompt before calling GPT-4o-mini. This instructs the LLM to generate output that is optimized for speech synthesis rather than reading:

```
## VOICE MODE
This response will be spoken aloud via text-to-speech. The TTS engine understands
natural language delivery cues in square brackets — use them anywhere in your
response
to control tone, pacing, and emotion.

- Keep your reply to 1-3 SHORT sentences max. Be punchy and concise.
- Do NOT include any action narration like *does something* — only spoken dialogue
and [cues].
- Place [cues] wherever they make sense — at the start, mid-sentence, between
sentences
  — to shift delivery naturally.

Examples:
  [gruff, dismissive] Yeah, I don't think so, pal.
  [excited whisper] Dude... [building intensity] I think that's a dragon.
  Well, [slow, ominous] you sure you wanna go down that road? [beat] Didn't think
so.
  [laughing] Ha! You remind me of my old adventuring buddy. [warmly] Good times.
```

The LLM then produces responses like:

> `[low, weary] Another adventurer. [pause] Try not to bleed on the floor this time.`

Gemini TTS natively interprets these bracketed cues as prosody and delivery instructions — they are not stripped before passing to TTS.

2c. Per-NPC Accent Prompts (TTS Input Engineering)

Each NPC has an optional `TTS_ACCENT` string stored in their character config. This string is prepended to every TTS input before it is sent to the model, anchoring the voice to a specific accent and speech style:

Brynleaf (dwarven ranger):

```
[thick, heavy Scottish highland brogue — think the dwarves from Lord of the Rings,
rough and weathered, speaks slow and deliberate with clipped consonants]
```

Kai (young fighter):

```
[loud, energetic young Chinese male voice — fast-talking, impulsive, always fired
up.
Sentences crash into each other. Talk like you're ready to fight RIGHT NOW.
High energy, competitive, no patience for waiting.]
```

Kumo (the blacksmith):

```
[deep, rumbling Japanese male voice — like distant thunder, speaks slowly and with
weight,
every word deliberate]
```

The full TTS input to Gemini becomes:

```
{TTS_ACCENT} {LLM-generated response with [cues]}
```

For example, Kai saying something might produce:

```
[loud, energetic young Chinese male voice — fast-talking, impulsive...]
[excited] Oh YEAH, that's what I'm talking about! [laughing] Let's go!
```

This double-layer approach — accent cue sets the baseline register, inline `[cues]` modulate moment-to-moment delivery — gives significantly more expressive and character-consistent output than either technique alone.

---

3. Evaluation

3a. STT Evaluation — Transcription Quality

The STT system was evaluated informally across input conditions relevant to the use case (mobile device, voice chat in a noisy environment).

Condition	Transcription Quality	Notes
Baseline: quiet room, clear speech	High	Near-perfect for common D&D vocabulary

Condition	Transcription Quality	Notes
Noisy environment (background music)	Medium	Occasional word errors, proper nouns suffer most
Non-standard D&D terms (e.g., "Okhan", NPC names)	Low–Medium	No custom vocabulary; model defaults to phonetically similar common words
Natural speech pace	High	Interim results display well in real time

Key finding: The `lang: 'en-US'` setting provides a reasonable baseline. The main failure mode is D&D-specific proper nouns (location names, NPC names) which the model has no domain knowledge of. Mitigation: players can correct the transcript before sending.

Latency: Final transcript typically arrives within 200–400ms of speech ending on a modern device.

3b. TTS Evaluation — Expressiveness and Character Consistency

Four conditions were compared:

Condition	Setup	Result
Baseline	Raw LLM text → TTS, no accent, no cues	Neutral robotic delivery, no character differentiation
Voice only	Pre-selected NPC voice (e.g., Fenrir for Nibby)	Some natural variation, but all NPCs sound similar in register
Accent cue only	`TTS_ACCENT` prepended, no inline cues	Consistent accent/register, but flat emotional delivery
Full system	`TTS_ACCENT` + `[cues]` from voice-mode LLM	Strongest character consistency, natural emotional range

Metrics used:

- *Character consistency* (subjective): Does the voice match the character's personality across multiple responses?
- *Emotional range*: Does delivery change meaningfully between an excited vs. cautious response?
- *Realism*: Does it sound like a person rather than a machine?
- *Latency*: Time from send to audio ready on client

Latency results:

Step	Typical Time
GPT-4o-mini response generation	800–1500ms
Gemini TTS generation	900–1800ms
Total (voice message end-to-end)	1.8–3.3 seconds

The latency is acceptable for this use case — players expect a slightly longer wait when explicitly requesting voice output via the `!voice` command.

3c. NPC Prompt Engineering Evaluation — Character Consistency

Condition	Character Consistency	In-world Accuracy
No system prompt	Very low — generic AI assistant behavior	None
SOUL.md only	Medium — personality present, lore missing	Low
+ CONTEXT.md + MEMORY.md	High — consistent personality and world knowledge	High
+ journal.md (recent events)	Very high — NPC recalls recent in-session events	Very high
+ Voice Mode block	High + speech-optimized output	High

The journal system is particularly impactful: without it, NPCs respond as if they have no memory of prior sessions. With it, they naturally reference past events players experienced, reinforcing immersion.

Summary

The system combines three AI models in a pipeline optimized for character-driven audio interaction:

1. **Web Speech API** (Google ASR) converts player speech to text
2. **GPT-4o-mini** generates in-character NPC dialogue, conditioned on personality documents and voice-mode delivery instructions
3. **Gemini 2.5 Flash TTS** speaks the NPC response in a character-specific voice and accent

The key insight is that quality in the final audio output is determined largely by the **prompt engineering upstream** — how the LLM is instructed to write for speech, and how the accent context is anchored at the TTS input layer — rather than by model selection alone.